Alexandre Schneider 6844 FDU MSACS

GitHub Repo: GitHub - alxschneider/DistributedDataAccessLab · GitHub

!Important!: I did not added appsettings to .gitignore for educational purposes

This project is a fully containerized e-commerce marketplace backend built with .NET 10. It is composed of six independent microservices, all orchestrated with Docker Compose. The goal is to demonstrate core microservices patterns: separate databases, inter-service HTTP communication, dependency injection, containerization, and independent deployability.

Key points

- 6 APIs: Customer, Product, Order, Cart, Payment, Notification
- Each service has its own SQLite database — no shared data store
- Everything runs inside Docker containers
- Linq/Lambda expressions for sql query

Disclaimer:

- Since this is an educational, non-production application, business rules were placed directly in the controller for simplicity. In a production scenario, I would follow a use-case driven approach, extracting them into a dedicated service layer to respect separation of concerns.

Architectural Decisions:

- Rest api calls using Http
- DI -> Controllers declare dependencies from constructor
- Interface -> Facilitation DIP (From Solid)
- Containerized: Each api with it own dockerfile
- Docker Compose for orchestration
- Auto-Migration on Startups (easy to test)
- Soft-Deletes instead of full delete

Database per service (SQLite for simplicity), since it's not a production environment. In a production scenario, the database choice would depend on each service's functional and non-functional requirements. For example, Order and Payment services handle

transactional data with strong consistency needs — PostgreSQL or SQL Server would be a better fit (probably would go PostreSQL to use JsonB fields). Product Service could benefit from a document store like MongoDB if the catalog grows with flexible schemas. For high-throughput write-heavy services like Notifications, something like Cassandra could make sense due to their horizontal scalability.

Project Requirements:

- You must implement four independent microservices three of those should be the
    - Product Service: Manage product catalog, price, and stock.
    - Customer Service: Manage customer information.
    - Order Service: Create and track orders.
- Order Service must validate customer and product existence using HttpClient.
- Provide a Dockerfile for each microservice.
- System must start using: docker compose up --build.
- Each service must have its own DbContext.
- Async EF Core methods must be used.
- Independent migrations per service.
- Separate database per service.

Functional Requirements (created by the Developer)

- Product Management — The system must allow creating, reading, updating, and deactivating products, including name, price, stock quantity, and discount percentage.
- Customer Management — The system must support registering and managing customers with distinct roles: Buyer, Seller, and Admin.
- Order Creation and Tracking — Buyers must be able to place orders containing multiple items. Orders must be queryable by buyer or seller.
- Customer Validation on Order — When an order is created, the Order Service must validate that the buyer exists and has the Buyer role by calling the Customer Service via HTTP.
- Product Validation on Order — When an order is created, the Order Service must validate product existence and available stock by calling the Product Service via HTTP.
- Automatic Stock Decrement — Upon successful order creation, the system must automatically decrement the product stock in the Product Service.
- Shopping Cart — Buyers must be able to add items to a cart, update quantities, remove items, and clear the cart entirely.

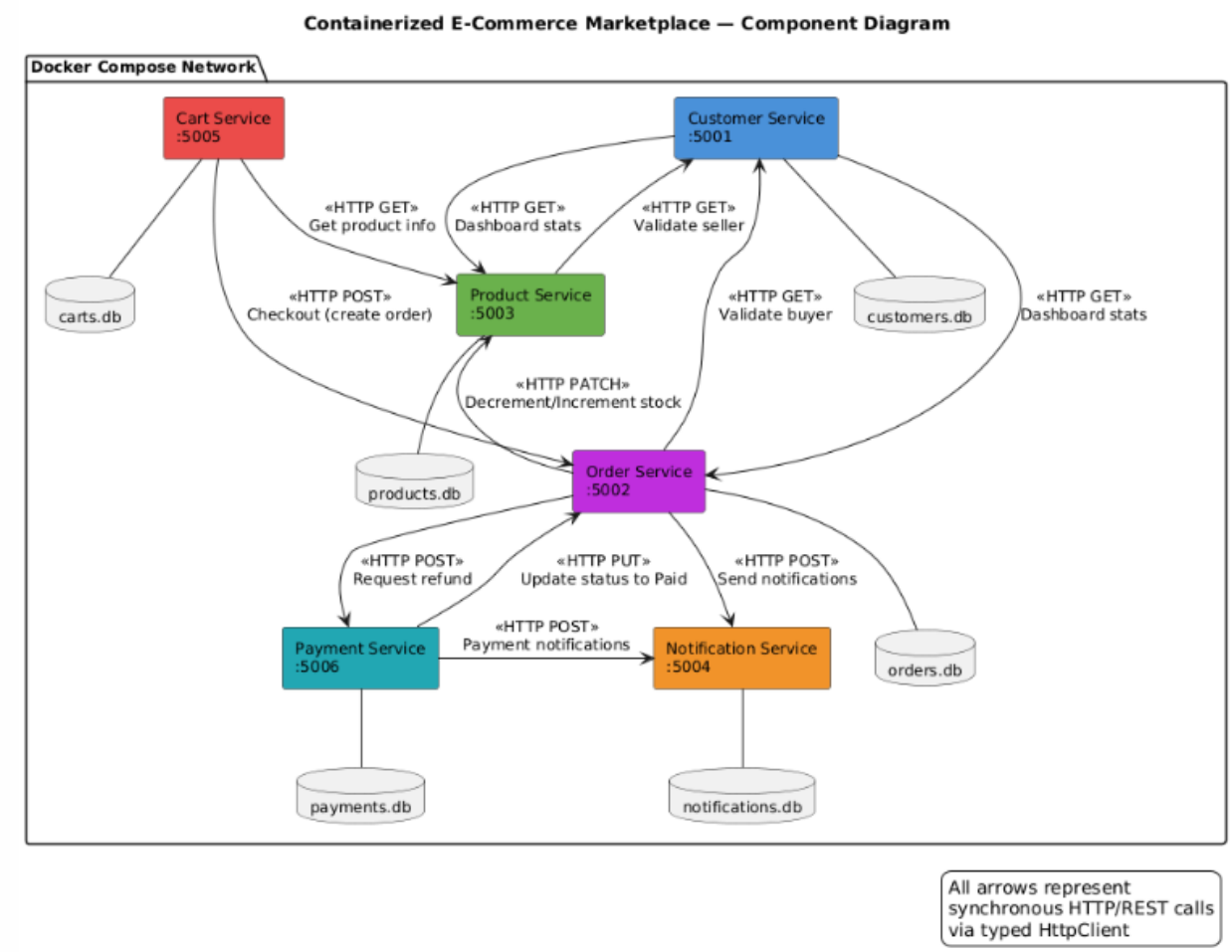
- Checkout — The Cart Service must orchestrate the checkout process by calling the Product Service (to validate stock) and the Order Service (to create the order), then clear the cart.
- Payment Processing — The system must allow creating payments linked to orders and processing them (approved or rejected), which updates the order status accordingly.
- Notifications — The system must generate notifications for buyers and sellers when key events occur, such as order creation, payment processing, and order cancellation.

Non-Functional Requirements (follow project requirements)

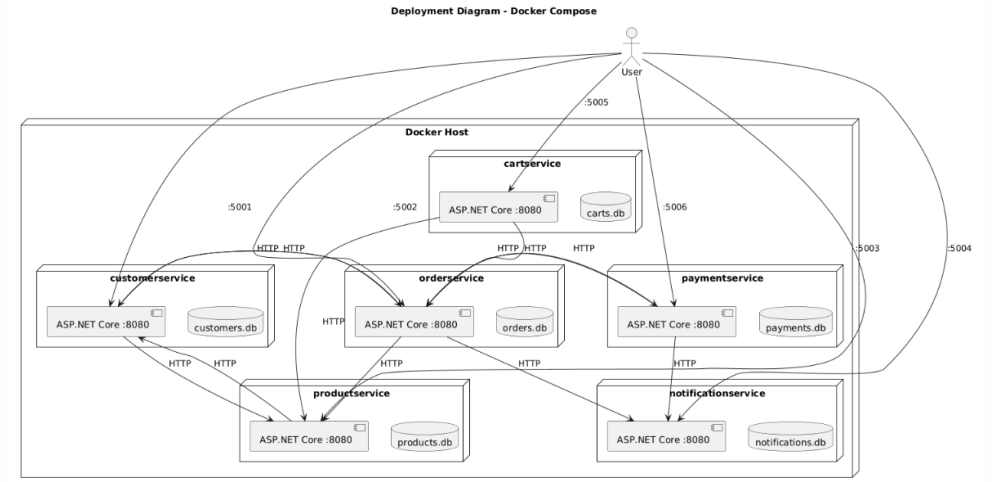
- Containerization — Each microservice must run in its own isolated Docker container, built using a multi-stage Dockerfile to minimize image size.
- Single-Command Startup — The entire system must start with a single docker compose up --build command.
- Database-per-Service — Each service must have its own independent SQLite database. No service may directly access another service's data store.
- Asynchronous Data Access — All database operations must use EF Core async methods (ToListAsync, SaveChangesAsync, FirstOrDefaultAsync, etc.) to avoid blocking threads.
- Independent Migrations — Each service must maintain its own EF Core migration history, allowing schema changes without affecting other services.
- Loose Coupling — Services must communicate exclusively through HTTP REST calls. No service may reference another service's code or database directly.
- Dependency Injection — All services must use ASP.NET Core's built-in DI container. Controllers must declare dependencies via constructor injection, programming against interfaces rather than concrete implementations.
- Service Discovery — Inter-service communication must use Docker Compose DNS-based service discovery, with service URLs configured via environment variables.
- Auto-Migration on Startup — Each service must automatically apply pending database migrations when the container starts, ensuring the schema is always up to date.
- Data Integrity — The system must use soft deletes (an IsActive flag) instead of physical deletion to preserve referential integrity and historical data.

Uml Diagrams

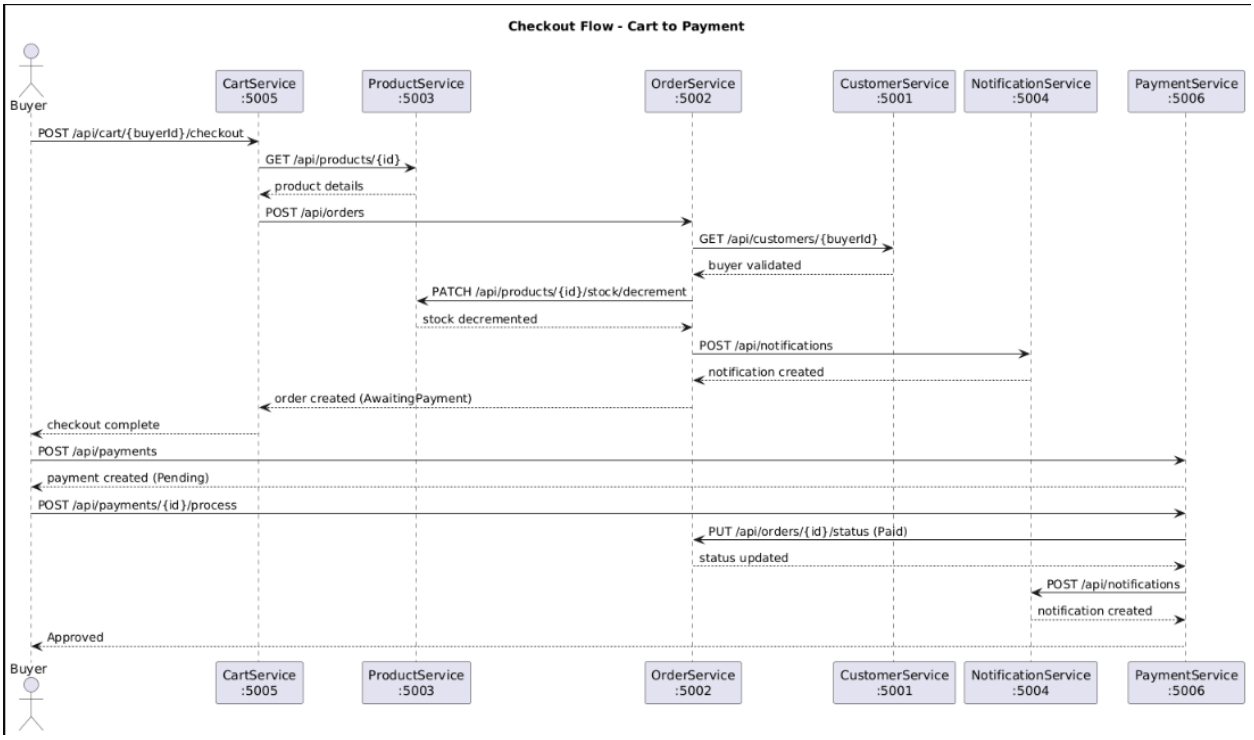
Component Diagram



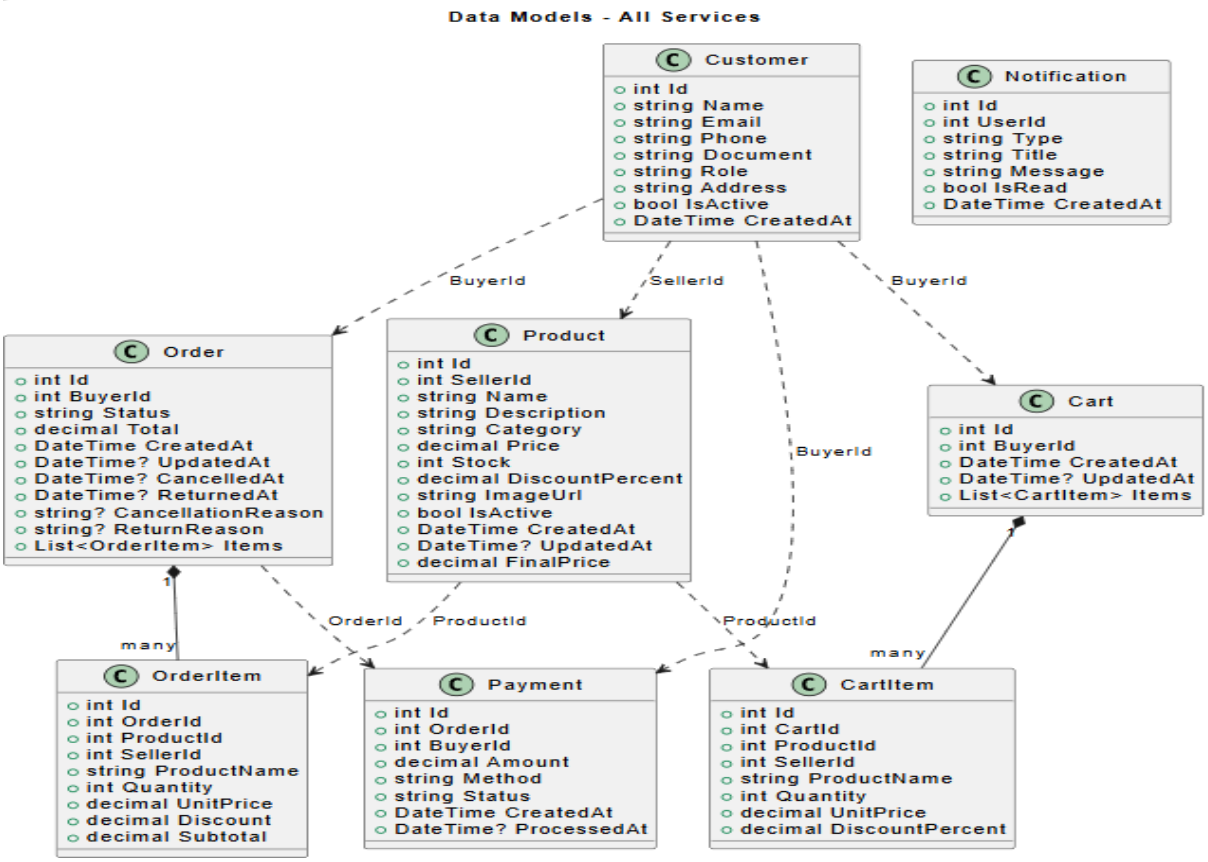
Deployment Diagram



Chechout Flow – Sequence Diagram



Data Model



Docker & Service Discovery

Each microservice has its own Dockerfile using multi-stage builds to keep images small. Everything is orchestrated with Docker Compose — one docker compose up --build starts all 6 services.

- Services communicate using Docker Compose's built-in DNS resolution. Each container name (like customerservice, productservice) works as a hostname inside the Docker network.
- Service URLs are configured through environment variables in docker-compose.yml, for example:
    - Services__OrderService: http://orderservice:8080/
    - Services__ProductService: http://productservice:8080/

- This means no service has hardcoded URLs in code — Program.cs reads from configuration and falls back to the Docker DNS name if nothing is set.
- If we needed to scale or replace a service, we'd only change the compose file, no code changes needed.
- Each service exposes a different external port (5001-5006) mapped to the internal 8080, so they can all be tested individually from the host machine.
- SQLite databases are persisted through Docker volumes (./CartService/CartService.Api/Data:/app/Data), so data survives container restarts.
- depends_on is used to control startup order — for example, Order Service waits for Customer, Product, and Notification services to start before booting.

Data Ownership

Following the requirements, each microservice has its own database (SQLite), and they don't query other databases directly. All cross-service data access happens through HTTP REST calls.

Order Service does not read from products.db — it calls GET /api/products/{id} via typed HttpClient to validate stock and price before creating an order.

Relevant product info (name, price, discount) is snapshotted into OrderItem at purchase time, so historical data stays accurate even if the product changes later.

Customer Service does not access orders.db — it calls the Order Service API to get buyer/seller dashboard stats.

Each service is the single source of truth for its own domain.

Coupling Prevention

No shared models — each service declares its own DTOs. For example, Order Service has a local ProductInfo class to deserialize Product Service responses, instead of referencing ProductService.Api.Models.

Using DIP as communication pattern — controllers depend on interfaces (IProductClient, ICustomerClient), not concrete implementations. This makes it easy to swap or mock dependencies.

Service discovery via Docker Compose DNS — service URLs are injected through environment variables (Services__ProductService: http://productservice:8080/), so no hardcoded addresses.

Logical foreign keys (like BuyerId in Order, SellerId in Product) are plain integers with no cross-service DB constraints. Validation happens at application level through HTTP calls.